IN PARTNERSHIP WITH PLYMOUTH UNIVERSITY

Name: Karunanayaka Jayawardhana
Student Reference Number: 10952756

Module Code: **PUSL3122**	Module Name: **HCI, Computer Graphics, and Visualisation**
Coursework Title: **Portfolio Coursework**	
Deadline Date: 19/03/2026	Member of staff responsible for coursework: Dr. Thaimur Bhakshi
Programme: BSc (Hons) Software Engineering (SE)	

Please note that University Academic Regulations are available under Rules and Regulations on the University website www.plymouth.ac.uk/studenthandbook.

Group work: please list all names of all participants formally associated with this work and state whether the work was undertaken alone or as part of a team. Please note you may be required to identify individual responsibility for component parts.

10955156 - *Asini Aththanayake*
10952760 - *Hewawasam Bathika*
10952715 - *Don Hettiarachchi*
10952746 - *Liyana Jayasinghe*
10952756 - *Karunanayaka Jayawardhana*
10952417 - *Janith Kalansooriya*

We confirm that we have read and understood the Plymouth University regulations relating to Assessment Offences and that we are aware of the possible penalties for any breach of these regulations. We confirm that this is the independent work of the group.

Signed on behalf of the group:

Individual assignment: ***I confirm that I have read and understood the Plymouth University regulations relating to Assessment Offences and that I am aware of the possible penalties for any breach of these regulations. I confirm that this is my own independent work.***

Signed :

Use of translation software: failure to declare that translation software or a similar writing aid has been used will be treated as an assessment offence.

I *have used/not used translation software.

If used, please state name of software……………………………………………………………

Overall mark _____% **Assessors Initials ______** **Date_________**

Table of Contents

List of Figures

Roles and Responsibilities

ID	Name	Role	Responsibilities
10952760	Don Hettiarachchi	Backend & Security Developer	Responsible for implementing secure user authentication (login/registration) and managing database sessions via Supabase.
10952760	Hewawasam Bathika	3D Interactive Systems Developer	Responsible for the core room configuration logic and the dynamic 3D furniture system (spawning, scaling, and coloring).
10955156	Asini Aththanayake	UI/UX Developer	Responsible for designing and implementing the responsive user menu, profile dropdowns.
10952746	Liyana Jayasinghe	3D Environment Developer	Responsible for programming the custom architectural elements, including wall generation, door placements, and snapping logic.
10952756	Karunanayaka Jayawardhana	Project Lead & Graphics Engine Developer	Responsible for building the main React-Three-Fiber 3D Scene, configuring the Orthographic/Perspective cameras, and managing global lighting and Toolbar.
10952417	Janith Kalansooriya	Frontend Developer	Responsible for developing the responsive Landing Page and ensuring accessibility.

Project Links

GitHub repository (source code) link: Github Repository Link

OneDrive video presentation link: Presentation Link

Chapter 01. Introduction

This report documents the design, implementation, and evaluation of the "Furniture Visualizer," a web-based application developed to enhance the in-store customer experience for a furniture retailer. The application serves as an interactive collaboration tool, allowing interior designers to work alongside customers to construct virtual 2D floor plans and instantly visualize them in a realistic 3D environment. By taking into account room dimensions, color schemes, and precise furniture scaling, the software enables customers to make informed spatial and aesthetic decisions with confidence.

1.1. Application Features

The Furniture Visualizer provides a seamless, user-centric environment for interior design. The core features of the implemented application include:

- **Dashboard Workspace:** A secure, authenticated portal for designers to create, rename, delete, and search through their saved design portfolio.
- **Seamless 2D/3D Toggling:** Users can easily switch between an orthographic 2D floor plan view (for layout precision) and a perspective 3D view (for spatial realism) using a unified rendering engine.
- **Interactive Object Manipulation:** Users can drag-and-drop furniture (e.g., chairs, beds, cupboards, sofas), rotate items, and snap them to a geometric grid.
- **Customization:** Real-time modification of room dimensions, floor colors, and individual furniture scaling and shading.
- **Accessibility & UI:** A modern, fully responsive interface featuring a "Dark Mode" toggle (accessible via a quick keyboard shortcut) to ensure optimal viewing comfort and accessibility.

1.2. Functional and Non-Functional Requirements

The development was guided by the initial client scenario, supplemented by further requirements gathered during our initial planning phase to ensure a robust user experience.

Functional Requirements:

1. The system must allow designers to securely log in to manage their portfolios.
2. The system must allow users to input and store specific room dimensions (width, length) and color schemes.
3. The system must allow users to add furniture items, scale them, and change their colors.
4. The system must provide both 2D layout creation and 3D visualization capabilities.
5. The system must allow users to quickly rotate furniture and snap items to a grid for precise alignment.
6. The system must allow designers to save, edit, and delete completed designs for future consultations.

Non-Functional Requirements:

1. **Usability:** The interface must be highly intuitive, utilizing clear UI conventions so that designers require minimal training to operate it alongside clients.
2. **Performance:** The transition between 2D and 3D views must render smoothly without noticeable lag, ensuring an uninterrupted consultation.
3. **Accessibility:** The application must support sufficient color contrast and provide a Dark Mode environment to accommodate different user visual preferences and lighting conditions.

4. **Error Prevention:** The system must provide clear feedback (via toast notifications) for actions like saving or deleting, and include Undo/Redo functionality to easily recover from layout mistakes.

Chapter 02. Methods and Technology

2.1. Platform, architecture, technology, coding details

The "Furniture Visualizer" was developed as a modern, responsive web application using an Agile methodology. To ensure a robust, scalable, and maintainable system, the application utilizes a layered architecture rather than a monolithic design. The system is strictly divided into three interacting layers: the Presentation Layer (UI/UX), the State Management Layer (Business Logic), and the Data Layer (Backend Services).

- **Technology Stack:**
    - **Frontend (Presentation):** Built using **React** and **Next.js** (App Router) for efficient routing and rendering. **Tailwind CSS** and **Shadcn UI** were utilized to create a consistent, accessible, and responsive interface that adheres strictly to HCI principles, including a dynamic Dark Mode toggle for visual comfort.
    - **Graphics Engine:** 3D rendering is powered by **Three.js**, implemented declaratively via **React-Three-Fiber** and @react-three/drei. This allows for seamless integration of WebGL computer graphics within the React component tree.
    - **State Management (Business Logic): Zustand** was selected to manage the complex global state. This decoupled layer handles the active room dimensions, arrays of furniture and walls, and the complex Undo/Redo history stack without forcing unnecessary UI re-renders.
    - **Backend (Data Layer): Supabase** manages secure user authentication (via Email OTP) and remote database storage, allowing designers to persist and retrieve their design portfolios.
- **Computer Graphics Algorithms & Visualization:** The core of the application relies on several fundamental computer graphics algorithms:
    - **Coordinate Transformations:** Matrix transformations are used extensively. When users interact with the UI sliders or the 3D Transform Gizmo, the application updates the translation (position), rotation (Euler angles), and scaling vectors of the specific mesh objects.
    - **Projection Algorithms:** To fulfill the requirement of seamless 2D/3D toggling, the engine dynamically swaps between an **Orthographic Camera** (for top-down, distortion-free 2D floor planning) and a **Perspective Camera** (providing depth and realistic 3D spatial awareness).
    - **Raycasting:** For 2D drag-and-drop functionality, raycasting algorithms calculate the intersection between the user's 2D screen mouse coordinates and the 3D mathematical plane of the room's floor, enabling precise translation of objects.
    - **Illumination and Shading:** The scene utilizes ambient lighting and directional lights with shadow mapping algorithms (castShadow and receiveShadow) to apply realistic shading to the room and furniture, fulfilling the brief's shading requirements.

2.2. Implementation and Testing

- **Implementation Evidence:**

Below are the key interfaces developed during the implementation phase. The code demonstrates clean separation of concerns, where UI components strictly observe the Zustand store for data changes.

- **The Designer Dashboard**

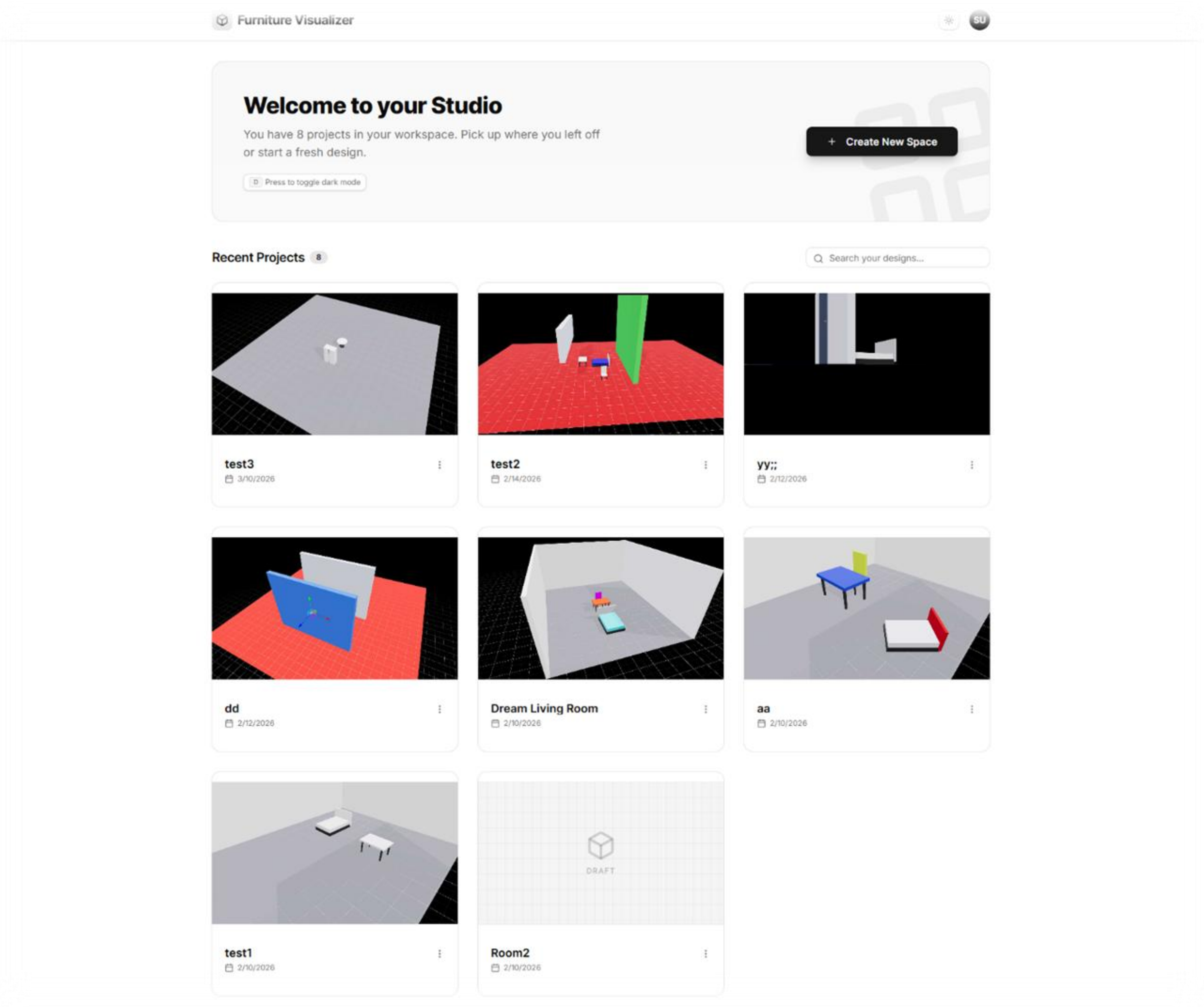

Figure 1: Dashboard View

The dashboard implements Supabase authentication and fetches the user's saved designs. It features a search filter and dynamic gallery rendering.

- **Source Code:** [Dashboard tsx file]

- **2D Floor Plan Editor**

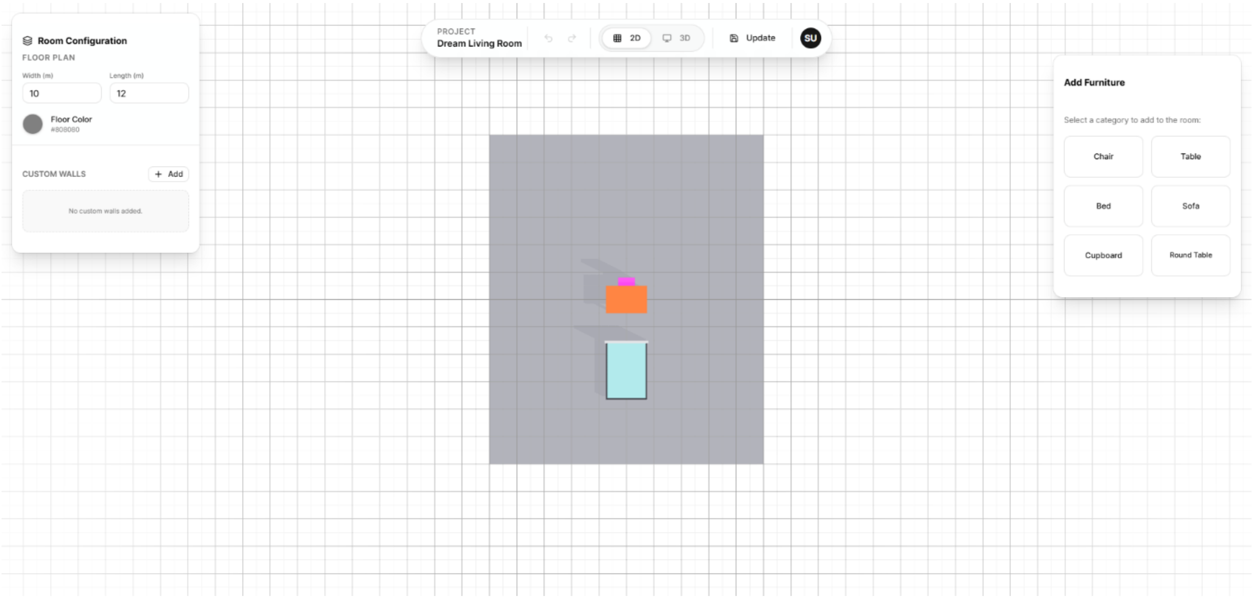

Figure 2: Editor 2D view

view utilizes the Orthographic camera. The properties panel allows for precise color changes and scaling of individual items.

  - **Source Code: The Rendering Layer (Three.js Canvas)**
  - **Source Code: The State Layer (Zustand Store)**
  - **Source Code: The UI Layer (Toolbar)**

- **3D Visualization and Lighting**

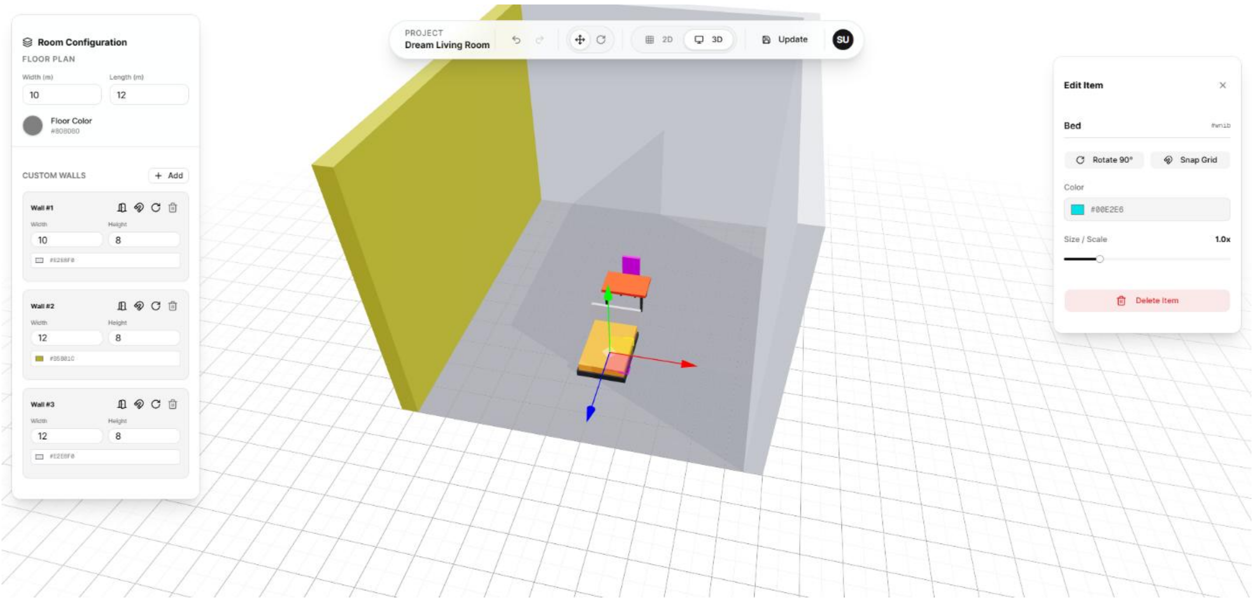

Figure 3: Editor 3D view and Gizmo controls

This demonstrates the application of shading, the 3D Transform Controls (Gizmo) for rotation, and the application of user-selected color hex codes to the standard mesh materials.

  - **Source Code: The Main Scene File**
  - **Source Code: The 3D Objects File**

- **Software Testing (Validation and Verification):** A highly competent testing system was implemented to ensure software reliability and usability.
    - **Validation (Building the right system):** Validation was continuously addressed through the HCI usability evaluations detailed in Chapter 1. Successive iterations of the UI were tested against the initial personas and client scenario to ensure the features (e.g., adding cupboards, round tables, changing room size) directly supported the interior designer's workflow.
    - **Verification (Building the system right):** System verification was conducted through rigorous manual testing of edge cases and state management logic.
        - Authentication Testing: Verified that incorrect OTPs throw appropriate error toasts and prevent access.
        - State Integrity: Thoroughly tested the Undo/Redo stack to ensure that rapid state changes (like dragging multiple furniture pieces) did not corrupt the history snapshot array.
        - Graphics Rendering: Tested camera boundary limits (maxPolarAngle, minDistance) within the OrbitControls to prevent users from clipping the camera through the floor or zooming infinitely into meshes, preventing graphical glitches.

Chapter 03. Limitations

The following table evaluates the final "Furniture Visualizer" application against the original functional requirements established in Appendix A of the coursework brief. While the application successfully meets the core objectives, a fair assessment of minor limitations is provided below.

Objective	**% completion**	**Comments**
Customer can provide the size, shape and colour scheme for the room.	90%	The user can easily define the size (width/length) and color scheme (wall/floor colors) via the Room Configuration panel. However, the "shape" is currently limited to standard rectangular rooms; complex polygonal room shapes were beyond the scope of this iteration.
Customer can create a new design based on the room size, shape and colour scheme.	100%	Users can successfully launch a new 3D workspace from the Dashboard that instantly applies their specified parameters to the rendering canvas.
Customer can visualise the design in 2D.	100%	Fully implemented using an Orthographic Camera in Three.js, allowing for an accurate, distortion-free top-down floor plan view.
Customer can visualise the design in 3D.	100%	Fully implemented using a Perspective Camera and OrbitControls, allowing the user to seamlessly rotate, pan, and zoom through the 3D space.

Customer can scale the design to best fit the room.	100%	Fully implemented. The Properties Panel includes a dynamic slider that uniformly scales the x, y, z vectors of the selected 3D mesh.
Customer can add shade to the design as a whole or selected parts.	100%	Fully implemented using WebGL rendering. A directional light source dynamically casts and receives shadows (castShadow, receiveShadow) across the floor and all furniture meshes in real-time.
Customer can change the colour of the design as a whole or selected parts.	100%	Fully implemented. Users can dynamically update the hexadecimal color values of both the global room (floor/walls) and individually selected furniture items via the Properties Panel.
Customer can edit/ delete the design.	100%	Fully implemented. The Dashboard allows users to rename, open (edit), or permanently delete their designs from the Supabase database.
Customer can save the design.	100%	Fully implemented. The toolbar includes a save/update function that securely serializes the room and furniture arrays (including positions and colors) into the PostgreSQL backend, alongside a generated thumbnail.

References

Cabello, R. (Mr.doob) and contributors, 2026. *Three.js - JavaScript 3D Library*. [online] Available at: https://threejs.org/ [Accessed 10 March 2026].

Figma, 2026. *Figma: The Collaborative Interface Design Tool*. [online] Available at: https://www.figma.com/ [Accessed 25 February 2026].

Meta Platforms, Inc., 2026. *React - A JavaScript library for building user interfaces*. [online] Available at: https://react.dev/ [Accessed 15 February 2026].

Poimandres, 2026. *React Three Fiber*. [online] Available at: https://docs.pmnd.rs/react-three-fiber/ [Accessed 1 March 2026].

Poimandres, 2026. *Zustand - Bear necessities for state management in React*. [online] Available at: https://docs.pmnd.rs/zustand/ [Accessed 1 March 2026].

Preece, J., Rogers, Y. and Sharp, H., 2015. *Interaction Design: Beyond Human-Computer Interaction*. 4th ed. Chichester: John Wiley & Sons.

Shadcn, 2026. *shadcn/ui - Beautifully designed components that you can copy and paste into your apps*. [online] Available at: https://ui.shadcn.com/ [Accessed 20 February 2026].

Supabase, 2026. *Supabase - The Open Source Firebase Alternative*. [online] Available at: https://supabase.com/ [Accessed 5 March 2026].

Vercel, 2026. *Next.js by Vercel - The React Framework*. [online] Available at: https://nextjs.org/ [Accessed 15 February 2026].

Appendix

Appendix 1: Usability Testing Participant Consent Form

Note: This form was provided to all participants prior to testing the Furniture Visualizer application to ensure ethical compliance.

Project Title: Furniture Visualizer Web Application

Module: PUSL3122 HCI, Computer Graphics and Visualization

Participant Declaration:

- I confirm that I am over the age of 18 (No children were involved in this study).
- I understand that my participation is entirely voluntary and I am free to withdraw at any time without giving a reason.
- I understand that all data collected during this evaluation will remain strictly anonymous and my identity will not be disclosed in the final report.
- I agree to take part in the usability testing (Think-Aloud protocol) for this software application.

Participant Signature: Date: 20/*02*/2026

Researcher Signature: Date: 20/*02*/2026

Appendix 2: Usability Evaluation Task List & Questionnaire

This script was used during the formative and summative testing phases with our participants.

Pre-Test Briefing:

"Thank you for participating. You will be testing a new web application designed for interior designers to visualize furniture in a room. Please remember we are testing the software, not you. If you get stuck, it is a flaw in our design, not your fault. Please 'think aloud' as you navigate."

Testing Tasks:

1. **Task 1 (Authentication):** Please create a new account or log in to the Dashboard.
2. **Task 2 (Creation):** Create a new 3D Space from the dashboard. Set the room dimensions to 15m x 15m.
3. **Task 3 (2D Layout):** Switch to the 2D floor plan view. Add a 'Sofa' and a 'Round Table' to the room.
4. **Task 4 (3D Manipulation):** Switch back to the 3D view. Rotate the Sofa by 90 degrees and change its color to a dark blue.
5. **Task 5 (Saving):** Save your project and return to the main Dashboard.

Post-Test Interview Questions:

1. Overall, how easy or difficult was it to navigate between the 2D and 3D views?
2. Did the application respond the way you expected when dragging and dropping the furniture?
3. Were the colors, contrast, and text readable? Did you try the Dark Mode feature?
4. What was the most frustrating part of using this application?
5. If you were an interior designer, what one feature would you add to this app?

Appendix 3: Raw User Feedback Notes

- **Participant A (Age 20):** Found the rotation tool slightly confusing at first in 3D mode, suggested we add clear keyboard shortcut hints. Loved the instant dark mode toggle.
- **Participant B (Age 23):** Successfully completed all tasks under 4 minutes. Suggested adding an Undo button (which we subsequently added to the final High-Fidelity prototype).